Práctica 1: Algoritmo A^*

1. Planteamiento del problema

El objetivo de esta práctica, para desarrollar en dos sesiones, es entender e implementar el algoritmo A^* para búsqueda en grafos dirigidos. El algoritmo deberá implementarse del modo más general posible, pero se probará aplicándolo a un juego llamado Takuzu o Binairo. Dicho juego se puede formular como sigue:

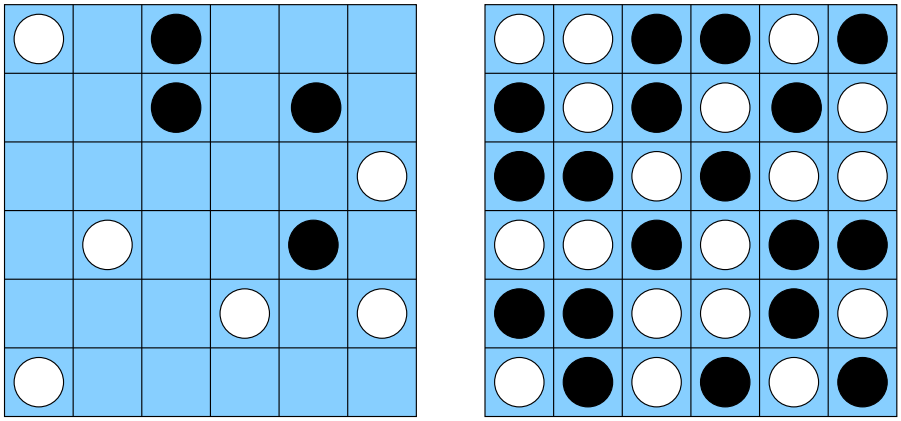
Sea un tablero cuadrado de $n \times n$ casillas donde n es un número par. En cada casilla del tablero puede haber una ficha blanca, una negra, o estar vacía. Se parte de una configuración inicial dada con algunas fichas ya colocadas que cumplen las reglas que detallaremos a continuación, y el único jugador debe ir colocando fichas, de cualquier color y en cualquier casilla a su elección, de modo que **en todo momento** se cumpla la siguiente regla:

> 1) No puede haber más de dos casillas consecutivas, ni en fila ni en columna, que tengan fichas del mismo color.

y además la configuración final debe cumplir estas reglas:

> 2) Todas las casillas deben estar ocupadas.
> 3) En cada fila y en cada columna debe haber el mismo número de casillas blancas y negras (que obviamente será $\frac{n}{2}$).
> 4) Ninguna fila ni columna puede repetirse.

Un ejemplo de un Takuzu de 6×6 en sus estados inicial y final sería:



Nótese que el jugador no puede situar una ficha digamos de color 'c' donde quiera, es decir, no todos los movimientos son legales: algunos violarían la regla 1, al situar tres fichas de color 'c' consecutivas. Otros violarían la regla 3, si ya hay $\frac{n}{2}$ fichas de color 'c' en esa fila/columna; y si falta una sola ficha para llenar una fila o columna, al completarla podría también violarse la regla 4.

Igualmente, nótese que situar una ficha en posición legal **obliga** a situar otras: si la ficha de color 'c' tiene junto a ella otra del mismo color, cualquier casilla vacía subsiguiente tendrá que llenarse con una ficha del color opuesto. Similarmente, si ya hay en una fila/columna $\frac{n}{2}$ fichas de color 'c', todas las casillas vacías de esa fila/columna deberán llenarse con el color opuesto. A esto lo llamaremos **movimientos obligatorios** y el programa los realizará automáticamente.

En nuestro formalismo cada estado del tablero será un **nodo** del grafo de estados. Los vecinos de ese nodo serán todos los estados que se generan al situar una ficha en cualquiera de las posiciones **legales** posibles **y completar con todos los movimientos obligatorios**. De este modo, el estado vecino a uno dado en el que había p fichas colocadas no necesariamente tiene que tener $p+1$ fichas, sino probablemente algunas o muchas más.

El programa procede generando todos los nodos vecinos del actual (en el sentido del párrafo anterior), evaluando cada uno con las funciones f y g cuyo significado es el dado en clase para el algoritmo A^*, y

aplicando dicho algoritmo para encontrar el nodo meta, que es aquel en el que todo el tablero está lleno. Vuestra tarea es programar el algoritmo A^* y probarlo con varias funciones heurísticas $h(n)$. Siempre h deberá tener un valor menor cuanto mejor (más deseable) consideréis el estado del tablero al que os lleva cierto movimiento.

2. Implementación

2.1. Formato de los datos

Se proporcionan algunos Takuzus para pruebas, que se dan como archivos de texto, de tamaños progresivamente mayores. Los nombrados como tnne son de tamaño nn y fáciles ('easy'); los nombrados tnnh son mas difíciles ('hard').

Al iniciar el juego el programa interno leerá estos archivos y los transformará a su representación interna (un objeto de la clase `Board`, ver apéndice). A partir de ahí, tenéis que resolver el problema usando el A^*, que programareis como una función, el cual generará la solución que debéis mostrar en pantalla junto con el número de iteraciones usado.

3. Entrega

Se entregará en un archivo comprimido:

- El código realizado SUFICIENTEMENTE COMENTADO. Incluid también el Makefile para compilarlo que ya se proporciona.
- Un archivo de texto LEAME con las instrucciones para compilar (generalmente, simplemente ejecutar make), algún comentario de comparación de los resultados con una u otra heurística y cualquier comentario adicional que queráis hacer.

ATENCIÓN: No se corregirán (y serán por tanto inválidas) las prácticas que ni siquiera compilen. Si compilan (en el sentido de generar un ejecutable) pero éste no hace nada que tenga sentido, la práctica estará probablemente suspendida.

4. Apéndice: API de C++

En el archivo `takuzu.h` se definen:

- El tipo de datos definido por enumeración `Token={e,b,w}` para indicar el posible contenido de una casilla (empty, black, white).
- El tipo de datos `Position` que es una estructura con dos campos de tipo `unsigned` llamados `row` y `col`, para representar las coordenadas de una casilla, que se cuentan desde 0.
- El tipo de datos `Move` que es una estructura con dos campos, uno de tipo `Position` llamado `sq` y otro de tipo `Token` llamado `value`, pera representar el movimiento consistente en poner una ficha de color `value` en la casilla `sq`.
- La clase `Board` para representar un estado del tablero (un nodo) y en ella dos estructuras de datos:
  `typedef list<Board> lb`
  para representar una lista de tableros, es decir, de nodos.
  `typedef list<pair<Move,Board> > lnei`
  para representar una lista de nodos vecinos a algún otro, donde el primer elemento del par representa el movimiento que nos llevará a cierto estado, y el segundo el estado al que llegamos al realizar efectivamente ese movimento, **y sus subsecuentes movimientos obligatorios**.

- La clase `Scores`, para guardar los valores de las funciones `f` y `g` de cualquier tablero.

Los constructores y funciones públicas (únicos que podéis usar) de la clase `Board` son:

`Board(unsigned size)` Constructor que crea un tablero vacío con `size` casillas de lado. `size` debe ser un número par.

`Board(string file_name)` Constructor que crea un tablero a partir del archivo de texto `file_name`, en el formato de los ejemplos que se entregan.

`Board(string file_name,ostream &outst)` Como el anterior, pero tras la carga del archivo muestra por pantalla (si se le pasa `cout`) o escribe en un archivo de texto abierto (si se le pasa el descriptor de archivo) el estado inicial, y el mismo tras realizar los movimientos obligatorios.

`unsigned GetSize()` Función que devuelve el número de casillas de lado del tablero.

`Token Content(Position p)` Función que devuelve el contenido de la casilla en la posición p.

`void Show(ostream &outst)` Procedimiento que muestra por pantalla (pasando `cout`) o escribe en un archivo de texto (pasándole el descriptor del archivo abierto) el tablero en su estado actual.

`void GetNeighbours(lnei &listnei)` Procedimiento que llena la variable `listnei` pasada por referencia con una lista de pares Movimiento/Tablero resultante para todos los movimientos legales que se pueden hacer desde el estado actual. Como se dijo, el tablero de la lista resulta de haber ejecutado no solo el movimiento, sino todos los obligados por él.

`bool Member(lb &listboards)` Función que devuelve `true` o `false` según si el tablero actual está o no en la lista de tableros que se pasa como argumento.

Por su parte, los constructores y funciones públicas (únicos que podéis usar) de la clase `Scores` son:

`void SetfValue(Board b,float v)` Procedimiento que almacena en la tabla de la función `f` el valor dado para el tablero que se le pasa.

`float GetfValue(Board b)` Función que devuelve el valor actualmente almacenado de `f` para el tablero que se le pasa, o $+\infty$ (en la práctica, la constante `FLT_MAX`) si aún no se ha fijado valor para ese tablero.

`void SetgValue(Board b,float v)` Procedimiento que almacena en la tabla de la función `g` el valor dado para el tablero que se le pasa.

`float GetgValue(Board b)` Función que devuelve el valor actualmente almacenado de `g` para el tablero que se le pasa, o $+\infty$ (en la práctica, la constante `FLT_MAX`) si aún no se ha fijado valor para ese tablero.

Un programa típico de prueba sería:

```
./takuzu_test t06h
```

que se llamaría desde la linea de órdenes pasando como argumento el archivo de entrada que contiene el takuzu propuesto.

Debéis implementar la función `bool AStar(...)` (pensad que parámetros se le deberían pasar) que devolverá `true` o `false` según si existe o no solución. También debéis implementar cualesquiera funciones auxiliares que consideréis necesarias Todo esto puede ir simplemente en el archivo principal, antes de `main`.

Y finalmente deben implementarse en el archivo `heuristic.cpp` las heurísticas, cuyo prototipo debe ser en cualquier caso el que se indica en `heuristic.h`, o sea:

```
float h(Board current_board,Move m,Board next_board)
```

Dentro de la función no es obligatorio usar sus tres parámetros; depende de la heurística que hayais pensado. Para escribir varias heurísticas y probarlas sucesivamente podéis escribir diferentes archivos `heuristic.cpp`, o ponerlas todas en el mismo, usando directivas condicionales de la forma

```
#ifdef nombre_HEURISTIC
(escribir la h..)
#endif
```

y usar el nombre que le hayais dado a la heurística como valor de la variable `CHOICE` que aparece al inicio del `Makefile`.